

Formalization of Fault Trees in Higher-order Logic: A Deep Embedding Approach^{*}

Waqar Ahmed and Osman Hasan

School of Electrical Engineering and Computer Science
National University of Sciences and Technology, Islamabad, Pakistan
{waqar.ahmad,osman.hasan}@seecs.nust.edu.pk

Abstract. Fault Tree (FT) is a standard failure modeling technique that has been extensively used to predict reliability, availability and safety of many complex engineering systems. In order to facilitate the formal analysis of FT based analyses, a higher-order-logic formalization of FTs has been recently proposed. However, this formalization is quite limited in terms of handling large systems and transformation of FT models into their corresponding Reliability Block Diagram (RBD) structures, i.e., a frequently used transformation in reliability and availability analyses. In order to overcome these limitations, we present a deep embedding based formalization of FTs. In particular, the paper presents a formalization of AND, OR and NOT FT gates, which are in turn used to formalize other commonly used FT gates, i.e., NAND, NOR, XOR, Inhibit, Comparator and majority Voting, and the formal verification of their failure probability expressions. For illustration purposes, we present a formal failure analysis of a communication gateway software for the next generation air traffic management system.

Keywords: Higher-order Logic, Fault Tree, Theorem Proving.

1 Introduction

Fault Tree (FT) is used as a standard failure modeling technique in various safety-critical domains, including nuclear power industry, civil aerospace and military systems. It mainly provides a graphical model for analyzing the conditions and factors causing an undesired top event, i.e., a critical event, which can cause the complete system failure upon its occurrence. The preceding nodes of the FT are represented by gates, like OR, AND and XOR, which are used to link two or more cause events of a fault in a prescribed manner. Using these FT gates, a FT model of a given system is constructed either on paper or by utilizing graphical editors provided by FT-based computer simulation tools, such as Relia-Soft [1] and ASENT [2]. In the paper-and-pencil proof methods, this obtained FT model is then used for the identification of the Minimal Cut Set (MCS) of failure events that are associated with the components of the given system. This is followed by associating the failure random variables, i.e., exponential or Weibull, to these MCS failure events. The Probabilistic Inclusion-Exclusion

^{*} The final publication is available at <http://link.springer.com>

(PIE) principle [3] is then used to evaluate the exact probability of failure of the overall system. On the other hand, the FT-based computer tools can be utilized to build a FT model by associating appropriate random variables with each component of the system. The reliability and the failure probability analysis of the complete system is then carried out by using computer arithmetic and numerical techniques on the generated samples from these random variables. However, both these methods cannot ascertain absolute correctness due to their inherent inaccuracy limitations. For instance, paper-and-pencil methods are prone to human errors, especially for large and complex systems, where a FT may consist of 50-130 levels of logic gates [4]. Manually manipulating such a large data makes it quite probable that some of MCS failure events may be overlooked, which would in turn lead to an erroneous design [4]. On the other hand, software tools can efficiently handle the analysis of large FTs but the computational requirements drastically increase as the size of the FT increases.

To overcome the above-mentioned limitations, a higher-order-logic formalization of some basic FT gates and their corresponding failure probability expressions [5] has been recently proposed. However, a major drawback of this formalization is the increase in complexity when analyzing FT of large and complex system. This formalization was primarily based on a shallow embedding approach, where the notion of each FT gate was explicitly defined on an event list and then its corresponding failure probability relationship was verified on the given failure event list. This approach makes the FT gate formalization non-compositional in nature, i.e., the basic FT gates, such as AND, OR and NOT, cannot be used to formalize other FT gates that are usually composed from these basic FT gates. Also, this work [5] utilizes the PIE principle to formally compute the exact failure probability of the given system, which limits its usability for complex system due to the involvement of large number of PIE terms. In the literature, several methods have been used to deal with this inherent complexity issue of the PIE principle. A tractable solution is to transform the given system FT to its equivalent Reliability Block Diagram (RBD) [6], which is also a well-known reliability modeling technique. This transformation considerably reduces the analysis complexity due to the fact that RBD offers closed form expressions compared to a FT, which requires unfolding of all the PIE terms.

In order to overcome the above-mentioned scalability issues of the existing formalization of FT gates [5] and thus broaden the scope of formal FT analysis, we propose a deep embedding approach to formalize the commonly used FT gates, such as AND, OR and NOT. This proposed formalization approach is compositional in nature and can be easily extended to formalize other FT gates, such as NAND, NOR, XOR, Inhibit, Comparator and majority Voting. It also enables us to transform the given system FT model to its equivalent RBD model, without any loss of valuable information. The RBD model can then be formally analyzed using our recently proposed formal reasoning support for RBDs [7].

To illustrate the practical effectiveness of our proposed approach, we present a formal failure analysis of a Next Generation (NextGen) Air Traffic Management (ATM) gateway system, which is primarily used to enhance the safety and

reliability of air transportation, to improve efficiency in the air transportation and to reduce aviation impact on the environment. The FT of the NextGen ATM gateway, which consists of more than 40 basic failure events including software, hardware, database update and transmission system is divided into four levels. The formally verified failure probability expressions of individual levels are then used to reason about the failure probability of the overall NextGen system. In addition, we also provide some automated reasoning support for the FT based failure analysis. This automation allows us to automatically simplify the failure expression of the NextGen system from the given values of the failure rates.

2 Related Work

The COMPASS tool-set [8] supports the dynamic FT analysis specifically for aerospace systems using the NuSMV and MRMC model checkers. The Interval Temporal Logic (ITS), i.e., a temporal logic that supports first-order logic, has been used, along with the Karlsruhe Interactive Verifier (KIV), for formal FT analysis of a rail-road crossing [9]. A deductive method for FT construction, in contrast to the intuitive approach followed in [9], by using the Observational Transition Systems (OTS), is presented in [10]. The formal analysis of this FT is then carried out using CafeOBJ [11], which is a formal specification language with interactive verification support. However, the scope of these tools is somewhat limited in terms of handling larger systems, due to the inherent state-space explosion problem of model checking. Moreover, either some of these approaches [9][10] do not cater for probabilities or if they do cater for them then the computation of probabilities in these methods [8] involves numerical techniques, which compromises the accuracy of the results.

Leveraging upon the high expressiveness of higher-order logic and the inherent soundness of theorem proving, Mhamdi’s formalized probability theory [12] has been recently used for the formalization of RBDs [7], including series [13], parallel [14], parallel-series [14] and series-parallel [15]. These formalizations have been used for the reliability analysis of many applications including simple oil and gas pipelines with serial components [13], wireless sensor network protocols [14] and logistic supply chains [14]. Similarly, Mhamdi’s probability theory have also been used for the formalization of commonly used FT gates, such as AND, OR, NAND, NOR, XOR and NOT, and the PIE principle [5]. In addition, the above-mentioned RBD and FT formalizations have been recently utilized for availability analysis [16]. In this paper, we have formalized the FT gates using a deep embedding approach to facilitate the analysis of larger FTs. Besides the existing formalization of FT gates [5], this paper also provides the formalization of inhibit, 2-bit comparator and Majority voting FT gates. Moreover, we have combined our existing formalizations of RBDs [13][14][15] to make the formal FT based analysis more scalable.

3 Probability Theory and Fault Trees in HOL

Mathematically, a measure space is defined as a triple (Ω, Σ, μ) , where Ω is a set, called the sample space, Σ represents a σ -algebra of subsets of Ω , where the subsets are usually referred to as measurable sets, and μ is a measure with

domain Σ . A probability space is a measure space (Ω, Σ, Pr) , such that the measure, referred to as the probability and denoted by Pr , of the sample space is 1. In the HOL4 formalization of probability theory [12], given a probability space p , the functions **space**, **subsets** and **prob** return the corresponding Ω , Σ and Pr , respectively. This formalization also includes the formal verification of some of the most widely used probability axioms, which play a pivotal role in formal reasoning about reliability properties.

A random variable is a measurable function between a probability space and a measurable space. The measurable functions belong to a special class of functions, which preserves the property that the inverse image of each measurable set is also measurable. A measurable space refers to a pair $(S, \mathcal{A})$, where S denotes a set and $\mathcal{A}$ represents a nonempty collection of sub-sets of S . Now, if S is a set with finite elements, then the corresponding random variable is termed as a discrete random variable otherwise it is called a continuous one.

The cumulative distribution function (CDF) is defined as the probability of the event where a random variable X has a value less than or equal to some value t , i.e., $Pr(X \leq t)$. This definition characterizes the distribution of both discrete and continuous random variables and has been formalized [13] as follows:

$\vdash \forall p \ X \ t. \text{CDF } p \ X \ t = \text{distribution } p \ X \ \{y \mid y \leq \text{Normal } t\}$

The function **Normal** takes a *real* number as its input and converts it to its corresponding value in the *extended-real* data-type, i.e, it is the *real* data-type with the inclusion of positive and negative infinity. The function **distribution** takes three parameters: a probability space $p : (\alpha \rightarrow \text{bool}) \# ((\alpha \rightarrow \text{bool}) \rightarrow \text{bool}) \# ((\alpha \rightarrow \text{bool}) \rightarrow \text{real})$, a random variable $X : (\alpha \rightarrow \text{extreal})$ and a set of *extended-real* numbers and returns the probability of the given random variable X acquiring all the values of the given set in probability space p .

The unreliability or the probability of failure $F(t)$ is defined as the probability that a system or component will fail by the time t . It can be described in terms of CDF, known as the failure distribution function, if the random variable X represent a time-to-failure of the component. This time-to-failure random variable X usually exhibits the exponential or Weibull distribution.

The notion of mutual independence of n random variables is a major requirement for reasoning about the failure analysis of most of the FT gates. According to this notion, a list of n events are mutual independent if and only if for each set of k events, such that $(1 \leq k \leq n)$, we have:

$$Pr\left(\bigcap_{i=1}^k A_i\right) = \prod_{i=1}^k Pr(A_i) \quad (1)$$

It is important to note that mutual independence is a much stronger property compared to pairwise independence [3], which ensures independence between two events only. On the other hand, mutual independence makes sure that any subset of events are independent with each other. Also, we can verify many interesting properties of independence using the mutual independence property. For instance, given a list of mutually independent events, say L , we can verify

that an element $h \in L$ is independent with the list $L - [h]$ representing the list L without element h .

The mutual independence concept is formalized in HOL4 as follows [13]:

```

⊢ ∀ p (L:α → bool). mutual_indep p L = ∀ L1 (n:num). PERM L L1 ∧
  1 ≤ n ∧ n ≤ LENGTH L ⇒
  prob p (inter_list p (TAKE n L1)) = list_prod (list_prob p (TAKE n L1))

```

The function `mutual_indep` accepts a list of events L and probability space p and returns *True* if the events in the given list are mutually independent in the probability space p . The predicate `PERM` ensures that its two lists as its arguments form a permutation of one another. The function `LENGTH` returns the length of the given list. The function `TAKE` returns the first n elements of its argument list as a list. The function `inter_list` performs the intersection of all the sets in its argument list of sets and returns the probability space if the given list of sets is empty. The function `list_prob` takes a list of events and returns a list of probabilities associated with the events in the given list of events in the given probability space. Finally, the function `list_prod` recursively multiplies all the elements in the given list of real numbers. Using these functions, the function `mutual_indep` models the mutual independence condition such that for n events taken from any permutation of the given list L , Equation (1) holds.

3.1 Formalization of Fault Tree Gates

The proposed formalization is primarily based on defining a new polymorphic datatype *gate* that encodes the notion of AND, OR and NOT FT gates. Then a semantic function is defined on that *gate* datatype yielding an event for the corresponding FT gate. This semantic function allows us to verify the generic failure probability expressions of the FT gates by utilizing the underlying probability theory within the sound core of the HOL4 theorem prover. Such a deep embedding considerably simplifies the FT gate modeling approach, compared to our previous work [5] (shallow embedding), and also enables us to develop a framework that can deal with arbitrary levels of FTs, which can be used to cater for a wide variety of real-world failure analysis problems.

We start the formalization process by type abbreviating the notion of event, which is essentially a set of observations with type `'a->bool` as follows:

```
type_abbrev ("event" , '':'a ->bool'')
```

We then define a recursive datatype *gate* in the HOL4 system as follows:

```
Hol_datatype 'gate = AND of gate list | OR of gate list | NOT of gate |
  atomic of 'a event'
```

The type constructors `AND` and `OR` recursively function on *gate*-typed lists and the type constructor `NOT` operates on *gate*-type variable. The type constructor `atomic` is basically a typecasting operator between *event* and *gate*-typed variables. These type constructors allow us to encode the notion of all the basic FT gates.

We define a semantic function $FTree : \alpha \text{ event} \# \alpha \text{ event event} \# (\alpha \text{ event} \rightarrow \text{real}) \rightarrow \alpha \text{ gate} \rightarrow \alpha \text{ event}$ over the above-defined *gate* datatype that can yield the corresponding event from the given FT gate as follows:

Definition 1: $\vdash (\forall p. FTree\ p\ (AND\ []) = p_space\ p) \wedge$
 $(\forall xs\ x\ p. FTree\ p\ (AND\ (x::xs)) = FTree\ p\ x \cap FTree\ p\ (AND\ xs)) \wedge$
 $(\forall p. FTree\ p\ (OR\ []) = \{\}) \wedge$
 $(\forall xs\ x\ p. FTree\ p\ (OR\ (x::xs)) = FTree\ p\ x \cup FTree\ p\ (OR\ xs)) \wedge$
 $(\forall p\ a. FTree\ p\ (NOT\ a) = p_space\ p \text{ DIFF } FTree\ p\ a) \wedge$
 $(\forall p\ a. FTree\ p\ (atomic\ a) = a)$

The above function decodes the semantic embedding of a FT by yielding a corresponding failure event, which can then be used to determine the failure probability of a given FT. The function **FTree** takes a list of type *gate*, identified by a type constructor **AND**, and returns the whole probability space if the given list is empty and otherwise returns the intersection of the events that are obtained after applying the function **FTree** on each element of the given list in order to model the AND FT gate behaviour. Similarly, to model the behaviour of the OR FT gate, the function **FTree** operates on a list of datatype *gate*, encoded by a type constructor **OR**. It then returns the union of the events after applying the function **FTree** on each element of the given list or an empty set if the given list is empty. The function **FTree** takes a type constructor **NOT** and returns the complement of the failure event obtained from the function **FTree**. The function **FTree** returns the failure event using the type constructor **atomic**.

If the occurrence of the failure event at the output is caused by the occurrence of all the input failure events then this kind of behavior can be modeled by using the AND FT gate. The failure probability expression of the AND FT gate can be expressed mathematically as follows:

$$F_{AND\text{-}gate}(t) = Pr(\bigcap_{i=2}^N A_i(t)) = \prod_{i=2}^N F_i(t) \quad (2)$$

Using Definition 1, we can verify the above equation in HOL4 as follows:

Theorem 1: $\vdash \forall p\ L. \text{prob_space}\ p \wedge$
 $(\forall x'. \text{MEM}\ x'\ L \Rightarrow x' \in \text{events}\ p) \wedge 2 \leq \text{LENGTH}\ L \wedge$
 $\text{mutual_indep}\ p\ L \Rightarrow$
 $(\text{prob}\ p\ (FTree\ p\ (AND\ (\text{gate_list}\ L)))) = \text{list_prod}\ (\text{list_prob}\ p\ L)$

The first two assumptions, in Theorem 1, ensures that p is a valid probability space and each element of a given event list L must be in event space p based on the probability theory in HOL4 [12]. The function **MEM** finds an element in a given list and returns false, if a match does not occur. The next two assumptions guarantee that the list of events L , representing the failure probability of individual components, must have at least two events and the failure events are mutually independent. The conclusion of the theorem represents Equation (2). The function **gate_list** generates a list of type *gate* by mapping the function **atomic** to each element of the given event list L to make it consistent with the assumptions of Theorem 1. It can be formalized in HOL4 as: $\forall L. \text{gate_list}\ L = \text{MAP}\ (\lambda a. \text{atomic}\ a)\ L$

The proof of Theorem 1 is primarily based on a mutual independence property and some fundamental axioms of probability theory.

In the OR FT gate, the occurrence of the output failure event depends upon the occurrence of any one of its input failure event. Mathematically, the failure probability of an OR FT gate can be expressed as:

$$F_{OR_gate}(t) = Pr(\bigcup_{i=2}^N A_i(t)) = 1 - \prod_{i=2}^N (1 - F_i(t)) \quad (3)$$

By following the approach, used in Theorem 1, we can formally verify the failure probability expression OR FT gate, given in Equation (3), in HOL4:

Theorem 2: $\vdash \forall p L. \text{prob_space } p \wedge 2 \leq \text{LENGTH } L \wedge$
 $(\forall x'. \text{MEM } x' L \Rightarrow x' \in \text{events } p) \wedge \text{mutual_indep } p L \Rightarrow$
 $(\text{prob } p (\text{FTree } p (\text{OR } (\text{gate_list } L)))) =$
 $1 - \text{list_prod } (\text{one_minus_list } (\text{list_prob } p L)))$

The above theorem is verified under the same assumptions as Theorem 1. The conclusion of the theorem represents Equation (3) where, the function `one_minus_list` accepts a list of *real* numbers $[x_1, x_2, x_3, \dots, x_n]$ and returns the list of *real* numbers such that each element of this list is 1 minus the corresponding element of the given list, i.e., $[1 - x_1, 1 - x_2, 1 - x_3, \dots, 1 - x_n]$.

The NOT FT gate can be used in conjunction with the AND and OR FT gates to formalize other FT gates. The formalization of these gates is given in Table 1. The NAND FT gate, represented by the function `NAND_FT_gate` in Table 1 models the behavior of the occurrence of an output failure event when at least one of the failure events at its input does not occur. This type of gate is used in FTs when the non-occurrence of the failure event in conjunction with the other failure events causes the top failure event to occur. This behavior can be expressed as the intersection of complementary and normal events, where the complementary events model the non-occurring failure events and the normal events model the occurring failure events. The output failure event occurs in the 2-input XOR FT gate if only one, and not both, of its input failure events occur. The inhibit FT gate produces an output failure event only if the conditional event occurs at the same time when the input failure event occurs. The HOL4 function `inhibit_FT_gate`, given in Table 1 models the behavior of a 2-input inhibit FT gate by composing the type constructors `AND`, `OR` and `NOT`. In the comparator FT gate, the output failure event occurs if all the failure events at its input occur or if all of the them do not occur. In the majority voting gate, the output failure event occurs if at least m out of n input failure events occurs. This behaviour can be modeled by utilizing the concept of binomial trials, which are used to find the chances of at least m success in n trials. The function `major_voting_FT_gate` accepts a probability space p , a binomial random variable X and two variables, m and n , which represent the number of successes and total number of trials, respectively. It then returns the union of the corresponding events that are associated with the binomial random variable X , which takes values from the set $\{x \mid k \leq x \wedge x < \text{SUC } n\}$. The function `IMAGE` takes a function f and an arbitrary domain set and returns a range set by applying the function f to all the elements of the given domain set. The function `BIGUNION` returns the union of all the element of given set of sets.

The verification of the corresponding failure probability expressions, of the above-mentioned FT gates, is presented in Table 2. These expressions are verified under the same assumptions as the ones used for Theorems 1 and 2. However, some additional provisos are required for the verification of majority voting gate as follows: (i) `prob_space` ensures that p is a valid probability space; (ii) $m \leq n$ makes sure that the number of successes of trails m must be less than or equal the total number of trials n ; (iii) $(\lambda x.$

Table 1: HOL4 Formalization of Fault Tree Gates

FT Gates	Formalization
	$\vdash \forall p \text{ L1 L2. NAND_FT_gate } p \text{ L1 L2} =$ $\text{FTree } p \text{ (AND (gate_list (compl_list } p \text{ L1 ++ L2)))}$
	$\vdash \forall p \text{ L. NOR_FT_gate } p \text{ L} = \text{FTree } p \text{ (NOT (OR (gate_list L)))}$
	$\vdash \forall p \text{ A B. XOR_FT_gate } p \text{ A B} =$ $\text{FTree } p \text{ (OR [AND [NOT A; B]; AND [A; NOT B]])}$
	$\vdash \forall p \text{ A B C. inhibit_FT_gate } p \text{ A B C} =$ $\text{FTree } p \text{ (AND [OR [A; B]; NOT C])}$
	$\vdash \forall p \text{ A B. comp_FT_gate } p \text{ A B} =$ $\text{FTree } p \text{ (OR [AND [A; B]; NOR_FT_gate } p \text{ [A; B]])}$
	$\vdash \forall p \text{ X m n. major_voting_FT_gate } p \text{ X m n} =$ $\text{BIGUNION (IMAGE (\lambda x. PREIMAGE X \{Normal (\&x)\} \cap p_space p)}$ $\{x \mid k \leq x \wedge x < \text{SUC } n\})}$

$\text{PREIMAGE } X \text{ Normal}(\&x) \cap p_space p) \in ((\text{count } (\text{SUC } n)) \rightarrow \text{events } p)$ ensures that all the corresponding events that are associated with the binomial random variable X are drawn from the events space p ; and (iv) $(\forall x. \text{distribution } p \text{ X } \{\text{Normal } (\&x)\} = (\&\text{binomial } n \text{ x}) * (F \text{ pow } x) * (1 - F) \text{ pow } (n-x))$ guarantees that the random variable X is exhibiting the binomial distribution.

3.2 Formalization of Probabilistic Inclusion-Exclusion Principle

In FT analysis, firstly all the basic failure events are identified that can cause the occurrence of the system top failure event. These failure events are then combined to model the overall fault behavior of the given system by using the fault gates. These combinations of basic failure events, called cut sets, are then reduced to minimal cut sets (MCS) by using some set-theory rules, such as idempotent, associative and commutative. Then, the Probabilistic Inclusion Exclusion (PIE) principle is used to evaluate the overall failure probability of the given system based on the MCS events. According to the PIE principle, if A_i represents the i^{th} basic failure event or a combination of failure events then the overall failure probability of the given system can be expressed as follows:

$$\mathbb{P}\left(\bigcup_{i=1}^n A_i\right) = \sum_{t \neq \{\}, t \subseteq \{1, 2, \dots, n\}} (-1)^{|t|+1} \mathbb{P}\left(\bigcap_{j \in t} A_j\right) \quad (4)$$

The above equation has been formally verified in HOL as follows [5]:

Theorem 3: $\vdash \forall p \text{ L. prob_space } p \wedge (\forall x. \text{MEM } x \text{ L} \Rightarrow x \in \text{events } p) \Rightarrow$
 $(\text{prob } p \text{ (union_list L)} =$
 $\text{sum_set } \{t \mid t \subseteq \text{set L} \wedge t \neq \{\}\}$
 $(\lambda t. -1 \text{ pow } (\text{CARD } t + 1) * \text{prob } p \text{ (BIGINTER } t)))$

The assumptions of the above theorem are the same as the ones used in Theorem 1. The function `sum_set` takes an arbitrary set s with element of type α and a real-valued function f and recursively sums the return values of the function f , when applied on each element of the given set s . In the above theorem, the set s is represented by the term $\{x \mid C(x)\}$ that contains all the values of x , which satisfy

Table 2: Probability of Failures of Fault Tree Gates

Mathematical Expressions	Theorem's Conclusion
$F_{NAND}(t) = Pr(\bigcap_{i=2}^k \bar{A}_i(t) \cap \bigcap_{j=k}^N A_j(t))$ $= \prod_{i=2}^k (1 - F_i(t)) * \prod_{j=k}^N (F_j(t))$	$\vdash \forall p \text{ L1 L2. } (\text{prob } p \text{ (NAND_FT_gate } p \text{ L1 L2)} =$ $\text{list.prod } ((\text{list.prob } p \text{ (compl_list } p \text{ L1)))} * \text{list.prod } (\text{list.prob } p \text{ L2}))$
$F_{NOR}(t) = 1 - F_{OR}(t) = \prod_{i=2}^N (1 - F_i(t))$	$\vdash \forall p \text{ L. } (\text{prob } p \text{ (NOR_FT_gate } p \text{ L)} =$ $\text{list.prod } (\text{one_minus_list } (\text{list.prob } p \text{ L})))$
$F_{XOR}(t) = Pr(\bar{A}(t)B(t) \cup A(t)\bar{B}(t))$ $= (1 - F_A(t))F_B(t) + F_A(t)(1 - F_B(t))$	$\vdash \forall p \text{ A B. prob_space } p \wedge A \in \text{events } p \wedge B \in \text{events } p$ $(\text{prob } p \text{ (XOR_FT_gate } p \text{ (atomic } A) \text{ (atomic } B))} =$ $(1 - \text{prob } p \text{ A}) * \text{prob } p \text{ B} + \text{prob } p \text{ A} * (1 - \text{prob } p \text{ B})$
$F_{inhibit}(t) = Pr((A(t) \cup B(t)) \cap \bar{C}(t))$ $= (1 - (1 - F_A(t)) * (1 - F_B(t))) * (1 - F_C(t))$	$\vdash \forall p \text{ A B C. } (\text{prob } p$ $(\text{inhibit_FT_gate } p \text{ (atomic } A) \text{ (atomic } B) \text{ (atomic } C)) =$ $(1 - (1 - \text{prob } p \text{ A}) * (1 - \text{prob } p \text{ B})) * (1 - \text{prob } p \text{ C})$
$F_{comp}(t) = Pr((A(t) \cap B(t)) \cup (\bar{A}(t) \cup \bar{B}(t)))$ $= (1 - (1 - F_A(t))F_B(t)) * (1 - (1 - F_A(t)) * (1 - F_B(t)))$	$\vdash \forall p \text{ A B C. } (\text{prob } p \text{ (comp_FT_gate } p \text{ (atomic } A) \text{ (atomic } B))} =$ $(1 - (1 - \text{prob } p \text{ A} * \text{prob } p \text{ B})) * (1 - (1 - \text{prob } p \text{ A}) * (1 - \text{prob } p \text{ B}))$
$F_{m n}(t) = Pr(\bigcup_{i=k}^n \{\text{exactly } i \text{ components are functioning properly}\})$ $= \sum_{i=m}^n \binom{n}{i} F^i (1 - F)^{n-i}$	$\vdash \forall p \text{ n k X F } (\text{prob } p \text{ (major_voting_FT_gate } p \text{ X m n)} =$ $\text{sum } (m, \text{SUC } n - m) (\lambda x. (\&\text{binomial } n \text{ x}) * (F \text{ pow } x) * (1 - F) \text{ pow } (n - x)))$

condition C . Whereas, the λ abstraction function $(\lambda t. -1 \text{ pow } (\text{CARD } t + 1) * \text{prob } p \text{ (BIGINTER } t))$ models $(-1)^{|t|+1} \mathbb{P}(\bigcap_{j \in t} A_j)$, such that the functions **CARD** and **BIGINTER** return the number of elements and the intersection of all the elements of the given set, respectively.

3.3 Formalization of Reliability Block Diagrams

Transformation of a system FT to its equivalent reliability block diagram (RBD) has been proposed as a viable solution to reduce the complexity associated with finding the failure probability of large systems [17]. The proposed deep embedding based formalization of FT gates allows the establishment of this link and thus we have used the existing formalization of RBDs [7] to make the formal analysis of FTs more scalable. In this paper, we only describe the formalization of the parallel-series RBD configuration because it is required to conduct the formal failure analysis of ASN gateway system, described in the next section.

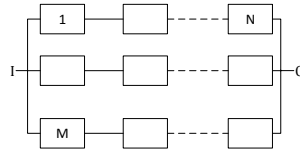


Fig. 1: Parallel-Series Reliability Block Diagrams

In a parallel-series RBD configuration, as shown in Fig. 1, the reserved *sub-systems* are connected serially and it can be considered as the nested form of series RBD in a parallel RBD configuration. If $A_{ij}(t)$ is the event corresponding to the reliability of the j^{th} component connected in a i^{th} subsystem at time t , then parallel-series RBD configuration can be expressed as:

$$R_{parallel-series}(t) = Pr(\bigcup_{i=1}^M \bigcap_{j=1}^N A_{ij}(t)) = 1 - \prod_{i=1}^M (1 - \prod_{j=1}^N (R_{ij}(t))) \quad (5)$$

The HOL4 formalization of the above equation is as follows [7]:

Theorem 4: $\vdash \forall p \text{ L. prob_space } p \wedge (\forall z. \text{MEM } z \text{ L} \Rightarrow \sim \text{NULL } z) \wedge$
 $(\forall x'. \text{MEM } x' (\text{FLAT L}) \Rightarrow x' \in \text{events } p) \wedge$
 $\text{mutual_indep } p (\text{FLAT L}) \Rightarrow$
 $(\text{prob } p (\text{rbd_struct } p ((\text{parallel of } (\lambda a. \text{series } (\text{rbd_list } a)))) \text{ L})) =$
 $(1 - \text{list_prod } (\text{one_minus_list}) \text{ of } (\lambda a. \text{list_prod } (\text{list_prob } p a))) \text{ L})$

where the function `rbd_struct` is defined on a recursive datatype *rbd* and can take any combination of type constructors `series` and `parallel`. It then yields the corresponding event of the given RBD configuration constituted by these type-constructors. The function `rbd_list` serves similar functionality as that of the function `gate_list`. The assumptions are quite similar to the ones used for Theorems 1 and 2. The conclusion models Equation (5) and the infix function `of` connects two *rbd* type-constructors by using the HOL4 `MAP` function.

4 Formalization of the NextGen ASN Gateway System

NextGen is supported by the nation-wide Aviation Simulation Network (ASN), which is an environment including simulated and human-in-the-loop (HIL) real-life components, e.g., pilots and air traffic controllers. The Real Time Distributed Simulation (RTDS) application suite [18] is used to facilitate the ASN by providing low and medium fidelity en-route simulation capabilities. An ASN gateway software system acts as an intermediary between RTDS and ASN by providing logic for data translation, two-way communication and transfer messages among them. The overall NextGen ASN gateway FT can be viewed as a four level FT [19]. The first or top level of the ASN gateway FT models an aviation accident caused by the lack of appropriate control, equipment, internal and external malfunctions. The internal failure event opens up to a second level of the ASN gateway FT, which comprises of failures related to the flight function mishap and transmissions. The flight mishap failure is caused by the failure of the Auto Pilot (AP) or Flight Director (FD) along with the failure not mitigated in time (FF1). The Transmission failure event captures the failure events due to data/message not correctly transmitted (A), failure to display (NotShown), and not performing transmission in a timely manner (RT). The third level of the ASN gateway FT is composed of several sub-FTs, given in Table 3, representing the RT and failure event A. The RT failure event occurs if the delay is too long for the transmission to meet its deadline (Time) and a latency problem occurs related to either the application (AL), serialization (SL), propagation delay (PD) or any other relevant sources. Similarly, the failure event A represents a failure

to correctly transmit a message and consists of two events. i.e., B1: failure to transfer a message from ASN to RTDS and B2: failure to transfer a message from RTDS to ASN of the communication link. The FT of the events B1 and B2 are given at the fourth level of the ASN gateway FT [19]. The overall ASN gateway FT consists of 47 basic failure events that are related to messages transmission failures, propagation delays, software and hardware equipment failures, database update failures and human mistakes.

4.1 Formal Fault Tree Models for ASN Gateway System

The formal definitions of FT gates [5] along with Definition 1 can be utilized to formally represent the FT of the ASN gateway in terms of its failure events. We systematically present the formalization of the ASN gateway FT by starting from the fourth level, i.e., the formalization of B1 sub-FT:

Definition 2: $\forall p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21.$
 $B1_FT \ p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21 =$
 $(OR \ [OR \ [atomic \ (fail_event \ p \ D1 \ t);$
 $\quad AND \ [OR \ (gate_list \ (fail_event_list \ p \ [E1; \ E2] \ t));$
 $\quad atomic \ (fail_event \ p \ E21 \ t)];$
 $\quad OR \ (gate_list \ (fail_event_list \ p \ [E3; \ E4; \ E5] \ t))]);$
 $OR \ [atomic \ (fail_event \ p \ D4 \ t);$
 $\quad AND \ [OR \ (gate_list \ (fail_event_list \ p \ [E6; \ E7] \ t));$
 $\quad atomic \ (fail_event \ p \ E21 \ t)];$
 $\quad OR \ (gate_list \ (fail_event_list \ p \ [E8; \ E9; \ E10] \ t))]])$

Where the random variables $D1$, $D4$, $E1 - E10$ and $E21$ model the time-to-failure of the communication process ASN to RTDS. The diagram of B1 FT is similar to B2 FT, which can be seen in Table 3. Additionally, the cut-set failure events in the above definition is already minimal, i.e., there are no combination of redundant failure events to be removed [19]. Therefore, the cut-sets and MCS for B1 sub-FT, in this case, are equivalent.

Similarly, other sub-FTs, such as B2-FT, A-FT, RT-FT and Internal-FT, which are at the fourth, third and second level of the ASN gateway FT can be formalized in HOL4 as shown in Table 3. It is important to note that the formal definition of the top level or first level FT, in Table 3, builds upon the formal definitions of all the other sub-FTs and models the complete ASN gateway FT.

We consider that the random variables, associated with the failure events of the ASN gateway FT, exhibit the exponential distribution:

Definition 3: $\vdash \forall p \ X \ l. \ exp_dist \ p \ X \ l =$
 $\forall x. \ (CDF \ p \ X \ x = \text{if } 0 \leq x \text{ then } 1 - \exp(-l * x) \text{ else } 0)$

The function `exp_dist` guarantees that the CDF of the random variable X is that of an exponential random variable with a failure rate l in a probability space p . We classify a list of exponentially distributed random variables as follows:

Definition 4: $\vdash \forall p \ L. \ list_exp \ p \ [] \ L = T \wedge$
 $\forall p \ h \ t \ L. \ list_exp \ p \ (h::t) \ L = exp_dist \ p \ (HD \ L) \ h \wedge list_exp \ p \ t \ (TL \ L)$
The function `list_exp` accepts a list of failure rates, a list of random variables L and a probability space p . It guarantees that all elements of the list L are exponentially distributed with the corresponding failure rates, given in the other

Table 3: ASN Gateway FT Levels with their HOL Formalizations

ASN Sub-FTs	Formal Definitions of Sub-FTs in HOL
	<pre> (B2.FT p t D7 D10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21) = OR [OR [atomic (fail.event p D7 t); AND [OR (gate_list (fail.event_list p [E11; E12] t)); atomic (fail.event p E21 t)]; OR (gate_list (fail.event_list p [E13; E14; E15] t))]; OR [atomic (fail.event p D10 t); AND [OR (gate_list (fail.event_list p [E16; E17] t)); atomic (fail.event p E21 t)]; OR (gate_list (fail.event_list p [E18; E19; E20] t))]] </pre>
	<pre> A_FT p t D1 D4 D7 D10 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21 C5 C6 C7 C8 = OR [B1_FT p t D1 D4 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E21; B2_FT p t D7 D10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21; AND [OR (gate_list (fail.event_list p [C5; C6; C7] t)); atomic (fail.event p C8 t)]] </pre>
	<pre> RT_FT p t AL SL PD Others time = OR_FT_gate [B1_FT p t D1 D4 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E21; AND [OR (gate_list (fail.event_list p [AL; SL; PD; Others] t)); atomic (fail.event p time t)]] </pre>
	<pre> Internal_FT p t FD AP FF1 D1 D4 D7 D10 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21 C5 C6 C7 C8 notshw AL SL PD Others time = OR [AND [OR (gate_list (fail.event_list p [FD; AP] t)); atomic (fail.event p FF1 t)]; OR [A_FT p t D1 D4 D7 D10 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21 C5 C6 C7 C8; notshw; RT_FT p t AL SL PD Others time]] </pre>
	<pre> ASN.gateway_FT p t FD AP FF1 D1 D4 D7 D10 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21 C5 C6 C7 C8 notshw AL SL PD Others time ED EQ1 EN1 EN2 EN3 EN4 human = OR [AND [OR (gate_list (fail.event_list p [FD; AP] t)); atomic (fail.event p FF1 t)]; AND [OR (gate_list (fail.event_list p [ED; EQ1] t)); OR [AND(gate_list (fail.event_list p [EN1; EN2; EN3; EN4] t)); fail.event p human t]; Internal_FT_gate p t FD AP FF1 D1 D4 D7 D10 E1 E2 E3 E4 E5 E6 E7 E8 E9 E10 E11 E12 E13 E14 E15 E16 E17 E18 E19 E20 E21 C5 C6 C7 C8 notshw AL SL PD Others time]] </pre>

list, within the probability space p . For this purpose, it utilizes the list functions HD and TL, which return the *head* and *tail* of a list, respectively.

4.2 Failure Assessment of NextGen ASN Gateway System

We now present the formal verification of all the sub-FTs, such as B1-FT, B2-FT, A-FT, RT-FT and Internal-FT. The formally verified results of these sub-FTs are then used to reason about the failure probability of overall ASN gateway communication system. Using the closed form expression of parallel-series RBD configuration, given in Equation (5), the failure probability of the B1-FT can be expressed mathematically as follows:

$$F_{B1}(t) = (1 - e^{-(c_1+c_2+c_3+c_4)t}) * (1 - (1 - e^{-C_{E1}t})(1 - e^{-C_{E21}t}))(1 - (1 - e^{-C_{E2}t})(1 - e^{-C_{E21}t}))(1 - (1 - e^{-C_{E6}t})(1 - e^{-C_{E21}t}))(1 - (1 - e^{-C_{E7}t})(1 - e^{-C_{E21}t})) \quad (6)$$

To verify Equation (6), we first verify a lemma that transforms the B1 sub-FT to its equivalent parallel-series RBD model as follow:

Lemma 1: $\vdash \forall p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21.$
 $FTree \ p \ (B1_FT \ p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21) =$
 $(rbd_struct \ p \ ((parallel \ of$
 $(\lambda a. \ series \ (rbd_list \ (fail_event_list \ a)))) \ [[D1]; [D4]; [E1; E21];$
 $[E2; E21]; [E3]; [E4]; [E5]; [E6; E21]; [E7; E21]; [E8]; [E9]; [E10]]))$

Now, using the formal definition of B1-FT and Lemma 1, the failure probability of B1 sub-FT can be verified in HOL4 as follows:

Theorem 5: $\vdash \forall p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21 \ C_E1 \ C_E2$
 $C_E6 \ C_E7 \ C_D1 \ C_D4 \ C_E3 \ C_E4 \ C_E5 \ C_E8 \ C_E9 \ C_E10 \ C_21.$
 $time_positive \ t \wedge prob_space \ p \wedge$
 $in_events \ p \ (fail_event_list \ p \ [D1; D4; E1; \dots; E10; E21] \ t) \wedge$
 $mutual_indep \ p \ (fail_event_list \ p \ [D1; D4; E1; \dots; E10; E21] \ t) \wedge$
 $list_exp \ p \ [C_D1; C_D4; C_E1; \dots; C_E10; C_E21] \ [D1; D4; E1; \dots; E10; E21] \Rightarrow$
 $(prob \ p \ (B1_FT \ p \ t \ D1 \ D4 \ E1 \ E2 \ E3 \ E4 \ E5 \ E6 \ E7 \ E8 \ E9 \ E10 \ E21) =$
 $1 - exp(-(t * list_sum \ [C_D1; C_D4; C_E3; C_E4; C_E5; C_E8; C_E9; C_E10])) *$
 $list_prod(one_minus_exp_prod \ t$
 $[[C_E1; C_E21]; [C_E2; C_E21]; [C_E6; C_E21]; [C_E7; C_E21]]))$

The function `exp` represents the exponential function. The function `list_sum` is used to sum all the elements of the given list of failure rates, the function `one_minus_exp` accepts a list of failure rates and returns a one minus list of exponentials and the function `one_minus_exp_prod` accepts a two dimensional list of failure rates and returns a list with one minus product of one minus exponentials of every sub-list. For example, `one_minus_exp_prod[[c1; c2; c3]; [c4; c5]; [c6; c7; c8]]`
 $x = [1 - ((1 - e^{-(c1)x}) * (1 - e^{-(c2)x}) * (1 - e^{-(c3)x})); (1 - (1 - e^{-(c4)x}) * (1 - e^{-(c5)x})); (1 - (1 - e^{-(c6)x}) * (1 - e^{-(c7)x}) * (1 - e^{-(c8)x}))]$. The first assumption ensures that the variable t models time t as it can acquire positive integer values only. The next assumption ensures that p is a valid probability space based on the probability theory in HOL [12]. The next two assumptions ensure that the

events corresponding to the failures modeled by the random variables D1, D2, E1 to E10 and E21 are valid events from the probability space p and they are mutually independent. Finally, the last assumption characterizes the random variables D1, D2, E1 to E10 and E21, as exponential random variables with failure rates C.D1, C.D2, C.E1 to C.E10 and C.E21, respectively. The conclusion of Theorem 5 represents the failure probability of the communication process between ASN to RTDS in terms of the failure rates of the components involved during the communication process. The proof of Theorem 5 is primarily based on Theorem 4 and some fundamental facts and axioms of probability.

Similarly, the failure probabilities of other sub-FTs, i.e., B1-FT, B2-FT, A-FT, RT-FT and Internal-FT, are verified in HOL4 [20]. These theorems are verified under the same assumptions as the one used in Theorem 5.

Now, using the formal definitions of ASN gateway sub-FTs, given in Table 3, and their verified failure probability results [20], we formally verified the failure probability of the complete ASN gateway system as follows:

Theorem 6: $\vdash (\text{prob } p \text{ (ASN_gateway_FT } p \text{ t FD AP FF1 D1 D4 D7 D10 E1 } \dots \text{ E21 C5 C6 C7 C8 notshw AL SL PD Others time ED EQ1 EN1 } \dots \text{ EN4 human) = } \\ 1 - (\text{list_prod}(\text{one_minus_exp_prod } t \text{ [[C_ED;C_EQ1];} \\ \text{[C_EN1;C_EN2;C_EN3;C_EN4]; [C_E6;C_E21]]})) * \\ \text{exp } (-(t * C_human)) * \text{exp } (-(t * C_notshw)) * \\ 1 - (\text{list_prod}(\text{one_minus_exp_prod } t \text{ [[C_FD;C_FF1]; [C_AP;C_FF1]]})) * \\ 1 - (1 - \text{exp}(-(t * \text{list_sum } [C_D1;C_D4;C_E3;C_E4;C_E5;C_E8;C_E9;C_E10]))) * \\ \text{list_prod}(\text{one_minus_exp_prod } t \text{ [[C_E1;C_E21]; [C_E2;C_E21];} \\ \text{[C_E6;C_E21]; [C_E7;C_E21]]})) * \\ 1 - \text{exp}(-(t * \text{list_sum } [C_D7;C_D10; C_E13;C_E14;C_E15;C_E18;C_E19;C_E20]))) * \\ \text{list_prod}(\text{one_minus_exp_prod } t \\ \text{[[C_E11;C_E21]; [C_E12;C_E21]; [C_E16;C_E21]; [C_E17;C_E21]]})) * \\ \text{list_prod}(\text{one_minus_exp_prod } t \text{ [[C_C5;C_C8];} \\ \text{[C_C6;C_C8]; [C_C7;C_C8]]})) * \\ \text{list_prod}(\text{one_minus_exp_prod } t \text{ [[C_AL;C_time];} \\ \text{[C_SL;C_time]; [C_PD;C_time]; [C_other;C_time]]}))$

The assumptions of the above theorem are similar to the ones used in Theorem 5 and its proof is based on Theorem 4 and some basic arithmetic lemmas and probability theory axioms. The proof of Theorems 5 and 6 and the formalization of sub-FTs, presented in Table 3, with their corresponding probability of failure took more than 2500 lines of HOL codes [20] and about 125 man-hours.

In order to facilitate the use of our formally verified results by industrial design engineers for their failure analysis, we have also developed a set of SML scripts to automate the simplification step of these theorems for any given failure rate list corresponding to the NextGen ATM system components. For instance, the output of the `auto_ASN_gateway_FT` script [20] for the automatic simplification of Theorem 6 is as follows:

$\vdash (\text{prob } p \text{ (ASN_gateway_FT } p \text{ t FD AP FF1 D1 D4 D7 D10 E1 } \dots \text{ E21 C5 C6 C7} \\ \text{C8 notshw AL SL PD Others time ED EQ1 EN1 } \dots \text{ EN4 human) = } \\ 1 - (1 - (1 - e^{(-5/2)}) * (1 - e^{(-3/2)})) * ((1 - (1 - e^{(-1/2)}) * ((1 - e^{(-2)}) * \\ ((1 - e^{(-3/2)}) * (1 - e^{(-4)})))) * e^{(-9/2)}) * ((1 - (1 - e^{(-7/2)}) * (1 - e^{(-3)})) * \\ (1 - (1 - e^{(-4)}) * (1 - e^{(-3)})) * (e^{(-4)} * ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) *$

$$\begin{aligned}
& ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * \\
& (1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})))) * (e^{(-321/20)} * ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * \\
& ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * \\
& (1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})))) * ((1 - (1 - e^{(-3/2)}) * (1 - e^{(-2)})) * \\
& ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-2)})) * (1 - (1 - e^{(-1/2)}) * (1 - e^{(-2)})))) * e^{(-1)} * \\
& ((1 - (1 - e^{(-7/2)}) * (1 - e^{(-3)})) * ((1 - (1 - e^{(-3/2)}) * (1 - e^{(-3)})) * \\
& ((1 - (1 - e^{(-1/2)}) * (1 - e^{(-3)})) * (1 - (1 - e^{(-5/2)}) * (1 - e^{(-3)}))))))
\end{aligned}$$

With a very little modification, these kind of automation scripts can facilitate industrial design engineers to accurately determine the failure probability of many other safety-critical systems.

5 Conclusion

The accuracy of failure analysis is a dire need for safety and mission-critical applications, like the avionic ASN gateway communication system, where a slight error in the failure analysis may lead to disastrous situations including the death of innocent human lives or heavy financial setbacks. In this paper, we presented a deep embedding based formalization of commonly used FT gates, which facilitates the transformation of a FT model to its equivalent RBD model. The transformation considerably reduces the complexity of the FT analysis compared to our earlier FT formalization [5]. For illustration, the paper presents the formalization of each level of ASN gateway FT and then building upon this formalization the failure probability of overall ASN gateways communication system is verified.

References

1. ReliaSoft: <http://www.reliasoft.com/> (2016)
2. ASENT: <https://www.raytheonagle.com/asent/rbd.htm> (2016)
3. Trivedi, K.S.: Probability and Statistics with Reliability, Queuing and Computer Science Applications. John Wiley and Sons Ltd. (2002)
4. Epstein, S., Rauzy, A.: Can we trust PRA? Reliability Engineering & System Safety **88**(3) (2005) 195–205
5. Ahmad, W., O.Hasan: Towards the Formal Fault Tree Analysis using Theorem Proving. In: Intelligent Computer Mathematics. Volume 9150 of LNAI., Springer (2015) 39–54
6. Bilintion, R., Allan, R.: Reliability Evaluation of Engineering Systems. Springer (1992)
7. Ahmed, W., Hasan, O., Tahar, S.: Formalization of Reliability Block Diagrams in Higher-order Logic. Journal of Applied Logic **18** (2016) 19–41
8. Bozzano, M., Cimatti, A., Katoen, J.P., Nguyen, V.Y., Noll, T., Roveri, M.: The COMPASS Approach: Correctness, Modelling and Performability of Aerospace Systems. In: Computer Safety, Reliability, and Security. Volume 5775 of LNCS. Springer (2009) 173–186
9. Ortmeier, F., Schellhorn, G.: Formal Fault Tree Analysis-Practical Experiences. Volume 185., Elsevier (2007) 139–151
10. Xiang, J., Futatsugi, K., He, Y.: Fault Tree and Formal Methods in System Safety Analysis. In: Computer and Information Technology, IEEE (2004) 1108–1115
11. Futatsugi, K., Nakagawa, A.T., Tamai, T.: CAFE: An Industrial-Strength Algebraic Formal Method. Elsevier (2000)

12. Mhamdi, T., Hasan, O., Tahar, S.: On the Formalization of the Lebesgue Integration Theory in HOL. In: Interactive Theorem Proving. Volume 6172 of LNCS. Springer (2011) 387–402
13. Ahmed, W., Hasan, O., Tahar, S., Hamdi, M.S.: Towards the Formal Reliability Analysis of Oil and Gas Pipelines. In: Intelligent Computer Mathematics. Volume 8543 of LNCS. Springer (2014) 30–44
14. Ahmed, W., Hasan, O., Tahar, S.: Formal Reliability Analysis of Wireless Sensor Network Data Transport Protocols using HOL. In: Wireless and Mobile Computing, Networking and Communications, IEEE (2015) 217–224
15. Ahmad, W., Hasan, O., Tahar, S., Hamdi, M.: Towards Formal Reliability Analysis of Logistics Service Supply Chains using Theorem Proving. In: Implementation of Logics. (2015) 111–121
16. Ahmed, W., Hasan, O.: Formal Availability Analysis using Theorem Proving. In: International Conference on Formal Engineering Methods. LNCS. Springer (2016) 1–16 To Appear, [arXiv:1608.01755](https://arxiv.org/abs/1608.01755)
17. Kuykendall, T.A.: Section 3.9, Fault Tree to RBD Transformation. In: Systems Engineering “Toolbox” for Design-Oriented Engineers. NASA (1994) 52–52
18. Törngren, M.: Fundamentals of Implementing Real-time Control Applications in Distributed Computer Systems. Real-time systems **14**(3) (1998) 219–250
19. Kornecki, A.J., Liu, M.: Fault Tree Analysis for Safety/Security Verification in Aviation Software. Electronics **2**(1) (2013) 41–56
20. Ahmad, W.: Formalization of Fault Trees in Higher-order Logic: A Deep Embedding Approach (2016) <http://save.seecs.nust.edu.pk/fault-tree/>